University of Oslo

Master's thesis

Master's Thesis

Grover's Algorithm and Quantum Error Correction

Emma Storberg

Computational Science: Physics
60 ECTS study points

Department of Physics
Faculty of Mathematics and Natural Sciences

UiO

Spring 2026

Emma Storberg

Master's Thesis

Grover's Algorithm and Quantum Error Correction

Supervisors:
Martin Strand, Thomas Gregersen, Joakim Bergli,
Nadia Slavila Larsen

Abstract

This thesis studies realistic resource requirements for applying Grover's algorithm to symmetric cryptanalysis under fault-tolerant assumptions. The work investigates how error-correcting architectures impact feasibility, with emphasis on quantum memory and logical operations. Implementations and simulations are developed for small code families, including the 5-qubit and 7-qubit codes, and exploratory analysis is performed for LDPC-style constructions such as surface and BB codes. The results provide practical insight into which design assumptions dominate runtime and overhead in near-term-to-fault-tolerant pathways.

Contents

1 Introduction 1

1.1 Quantum Prerequisities and Notation 4

2 Background 5

2.1 Jaques' Argument 6

2.2 Grover's Algorithm for Solving Unstructured Search 6

2.2.1 The Problem Definition 6

2.2.2 The Algorithm and Circuit 7

2.3 Quantum Error Correction 12

2.3.1 Stable Quantum Memory 13

2.4 Fault-tolerant Quantum Computation 18

2.4.1 Universal Gate Sets 18

3 Experiments and Analysis 23

3.1 General Limitations when Simulating Quantum Systems 24

3.1.1 Proof-of-concept Experiments with Smaller Codes 25

3.2 Utilizing Stabilizer Formalism as a Computational Tool 26

3.2.1 Software Aids 27

3.2.2 Simulating Stable Quantum Memory with PanQEC 27

3.2.3 Simulating Transversal Logical Operators with PanQEC . . . 29

3.3 The Trouble with the T-gate 29

3.3.1 Implemention 30

3.4 Assessing the Quantum Threat 31

3.4.1 T-gate Cost as a Bottleneck 31

3.4.2 Improvements with a Better Error Correcting Code 31

3.4.3 Grover's Algorithm as a Practical Threat 31

4 Conclusion 33

4.1 Future Work 34

4.2 Declaration of AI Usage 34

Appendices **35**

A Definitions of Smaller Error Correcting Codes Used 37

A.1 The 5-Qubit Code 37

A.2 The 7-Qubit Steane Code 38

B Verifying Transversality of Clifford Gates in Various Codes 39

B.1 Toric Surface Code 39

B.2 BB Code 39

List of Figures

2.1 A surface code grid with physical data qubits located on the edges, demonstrating a plaquette (Z stabilizer) in blue and a vertex (X stabilizer) in red. Although only one of each is shown here, the entire grid is covered in these stabilizers in a repeating pattern. Image source: Sayedsalehi et al. [20] WILL BE REPLACED WITH MY OWN MORE EXPLANATORY DIAGRAM. . 14

2.2 Illustration of transversality in a single- and multi-qubit gate. TO BE REPLACED WITH MY OWN DIAGRAM. 20

3.1 The smallest possible surface code consisting of nine data qubits. The indicated eight stabilizers could, in principle, be measured sequentially by a single additional qubit, or measured in parallel by an additional eight qubits. REMAKE MYSELF or ADD SOURCE TO BIBLIOGRAPHY. 25

3.2 Quantum memory simulation results comparing the BB code (orange) against surface code implementations at identical logical qubit counts (light blue), as well as matching code distance (dark blue). Logical error rate is plotted as a function of physical error rate under Monte Carlo sampling. 28

List of Tables

Chapter 1

Introduction

How should I structure the introduction besides the content I have now? I see some people have sections specifying notation, thesis structure, how to get access to code, data handling (although that's not relevant for me), and their declaration of AI usage may go here as well. I am worried splitting it up to much will make the narrative lose too much momentum, but if any of these things are nice for the reader to have, I will add them.

In recent years, quantum algorithms have garnered significant attention for their potential to disrupt traditional cryptographic systems. The security of many important communication channels, including the internet as we know it, is ensured by classical cryptographic protocols, particularly those based on the hardness of certain mathematical problems. With the rise of quantum technology, new algorithms have been found that exploit quantum mechanical principles, such as superposition and entanglement, to solve these problems exponentially faster than classical algorithms. Still, the implementation of quantum algorithms for cryptographic purposes continues to be largely theoretical. A quantum computer capable of breaking the most-used algorithms today is yet to be constructed, as there are many practical limitations to doing so. Thus, practical quantum computers, with thousands of qubits and low error rates, remain an area of active research; there is considerable anticipation surrounding the eventual realization of the theoretical promises associated with quantum computing. As we move into the post-quantum era, it will become increasingly important to assess our cryptographic protocols in light of the most recent developments in the field, in order to maintain and protect critical infrastructure and assets.

Brute-force search is a technique that may be employed to break cryptographic systems that use so-called *symmetric* keys, which means the the same keys for encryption and decryption [1], as opposed to *asymmetric* algorithms, which use separate encryption and decryption keys. Performing such a search involves quite simply checking every single possible key option until the right key is found, an inordinately time-consuming process if the key space is large enough. Our current protocols ensure security against these searches by having key lengths large enough that it is considered computationally infeasible for a classical computer to identify one in any reasonable amount of time. A closer look at the popular symmetric-key algorithm *Advanced Encryption Standard* (AES) [2] exemplifies this: It exists in versions that use key sizes of 128, 192 and 256 bits, resulting in search spaces of 2^{128}, 2^{192} and 2^{256} possible keys respectively.

A relevant quantum algorithm here is *Grover's algorithm* [3], in which a procedure is given for a quantum computer to perform a search much faster than brute-force search can be done on classical computers, thereby potentially threatening the security of AES. As we will see, Grover's algorithm offers a quadratic improvement over classical algorithms, meaning the number of operations needed when we use this quantum algorithm is on the order of the square-root of the number of operations required when solving the same

problem classically. However, finding the exact runtime is not as straightforward as the asymptotic estimates, which do not consider constant values, nor do they take into account what kind of parallelization may be possible. Analyzing the true cost of using Grover's algorithm in reality is a nontrivial problem that warrants further exploration, which constitutes the core of this project.

Building a quantum machine in practice is no easy feat. The root of the issue is that quantum information is extremely fragile. The qubits that store information are unstable and very vulnerable to noise that changes the value the qubit holds. We therefore wish to construct circuits of multiple qubits that work together to protect the information from errors, such that they form a single logical qubit that can have its errors located and corrected, making them more stable. This approach, though useful, will often require large numbers of qubits, which poses an engineering challenge to keep them all as stable as possible, facilitate connections between them, and perform computations on the logical qubits, rather than on the physical qubits individually.

Jaques considers this in his 2024 invited talk about Grover's algorithm as a quantum attack on AES [4], in which he suggests that due to the error correction models and gate sets currently in widespread use, a physical implementation of Grover's algorithm will never see the light of day on anything resembling current quantum architectures (or maybe ever). This project uses his argumentation as a starting point for further analysis, aiming to tackle some of the assumptions he makes and explore alternatives to some of the runtime bottlenecks he identifies.

Mention BB code. "since Jacques' argumentation bases itself so much on our continued usage of the surface code, what does his analysis look like under the assumption of a better error correcting code?"

Add a brief, non-technical note that classical simulation of quantum systems is intractable beyond small scale, motivating why this is hard to study experimentally, i.e. justifying my method.

A major problem with quantum computing research is that it is problematic to simulate quantum systems classically beyond small, trivial base cases; indeed, if we could simulate quantum systems without issue in general, it would beg the question why we need to realize them at all. This creates a limitation for us in studying Grover's algorithm, because it is hard to take circuit diagrams and see how it will work in practice. This project explores Grover's algorithm and what it can be used for to threaten cryptosystems and the security afforded by AES, what we would need to build it in practice and why it's so hard, why simulating the quantum hardware is equally as hard or harder, and covers the use of some methods and software we can use to research and simulate quantum systems.

Rewrite this paragraph and formalize a little; make sure it's not repetitive. Last sentence is just a

1.1 Quantum Prerequisities and Notation

Do I include this section?

Rewrite and formalize, if it is to be included at all.

We assume you know basic Dirac notation, maybe say that n is the number of qubits and N is 2^n?

Hadamard gate, superposition

Pauli matrices are the matrices I, X, Y and Z, defined as... Sometimes σ notation is used, but this text uses only the I, X, Y, Z notation (this is not true though, sigma notation shows up in surface code section about stabilizers).

The probability of measuring a particular component of a state vector in superposition is its complex scalar squared: "As we know, in a quantum state vector, the amplitude of any component squared represents the probability of seeing that component after measurement."

Chapter 2

Background

2.1 Jaques' Argument

Write Jaques' line of argumentation, making clear which aspects are in the scope of this project and which are not. Motivate the questions we have that lead to the code experiments we do.

here is another edit. does this change things?

Jaques' argument fundamentally relies on the assumption that we will encode the Grover circuit with the Clifford+T gate set, where the number of T-gates is ultimately the determining factor in his estimation of the time complexity of the algorithm. However, if there were a better way to create the circuit that requires less T-gates (or T-gate equivalents), we could possibly see substantial improvements, since the huge estimated cost of implementing T-gates is a major contributor to why Grover's algorithm works out so poorly in practice.

2.2 Grover's Algorithm for Solving Unstructured Search

Section 2.2 is adapted from parts of my previous work [5], with revisions and additions for clarity.

2.2.1 The Problem Definition

Formally, the computational problem that Grover's algorithm addresses is a quite general problem known as *unstructured search* [6]. It can be described as follows:

The unstructured search problem

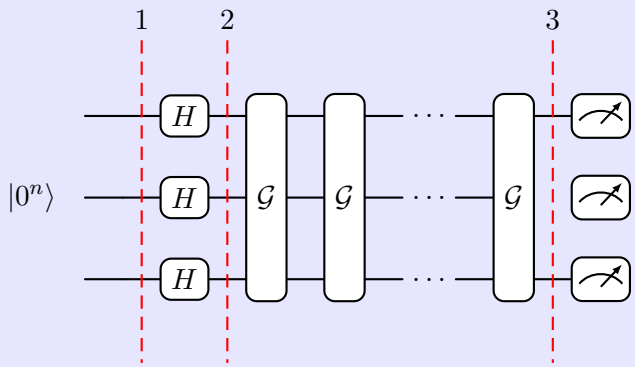
Let Σ denote the binary alphabet $\{0, 1\}$. Suppose we are given a function $f : \Sigma^n \to \Sigma$ that we can compute efficiently. We wish to determine the binary strings $x \in \Sigma^n$ such that $f(x) = 1$, or "no solution" if no such x exists.

We call this search problem *unstructured* because f can be an arbitrary function, meaning there is no underlying pattern or constraint, often called a *promise* [6], that we can take advantage of to systematically search for a solution string x, if one even exists. This means that in the classical approach to solving this problem, we can be sure of the solution only after checking each of the 2^n possible strings, thus requiring 2^n queries.

2.2.2 The Algorithm and Circuit

Circuit diagram of Grover's algorithm

The algorithm essentially consists of three main parts: we have initialization of n qubits into an equal superposition (1), followed by t Grover iterates (2), and finally measurement to reveal a candidate solution (3). These three stages are displayed and labeled in the circuit diagram below:



Let's dive into the algorithm. Our first step is using the *Hadamard gate* (denoted by H) [7] on each of the n input qubits. H has no classical equivalent, and using it allows us to take advantage of a strictly quantum phenomenon known as *superposition* [8]. This means the qubit can evaluate to either 0 or 1, and we can apply processing to it as if the qubit holds both values simultaneously. Thus, in finding a binary string of length n, using the Hadamard gate on $|0^n\rangle$ lets us evaluate all 2^n strings in Σ^n concurrently. For convenience, we will use the letter N to refer to the value 2^n from this point on. Next is the Grover iteration step, represented by sequential application of the Grover operator $\mathcal{G}$ in the diagram.

The Grover operation $\mathcal{G}$

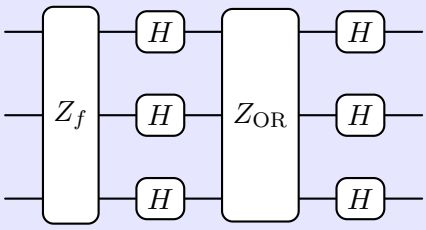
The gate sequence needed for the Grover operation $\mathcal{G}$ is:

$$H^{\otimes n} Z_{\mathrm{OR}} H^{\otimes n} Z_f$$

where Z_f is the phase query gate for the function f, and Z_{OR} is the phase query gate for the n-bit logical OR function. They are defined as follows:

$$Z_f : |x\rangle \mapsto (-1)^{f(x)} |x\rangle \qquad \forall x \in \Sigma^n$$
$$Z_{\mathrm{OR}} : \begin{cases} |x\rangle & x = 0^n \\ -|x\rangle & x \neq 0^n \end{cases}$$

Since the gates are applied from right to left, we apply Z_f first, followed by H, then Z_{OR}, and finally another set of Hadamard gates. The circuit diagram of the Grover operation on three bits looks like this:



Concerning evaluation of the circuit, only the Z_f gate requires a query to f. Thus, the number of iterations t we perform with $\mathcal{G}$ corresponds to how many queries to f we need.

A natural way to use the algorithm for search problems is to choose a number of iterations t of the Grover operation, run the circuit to obtain a candidate solution x, and check if x is a solution (in this case whether $f(x) = 1$). If so, we return x, otherwise we run the algorithm again (possibly with a different value for t) or return "no solution". This sounds simple enough, but still, important questions remain unanswered: What is the "suitable" number of operations t, how do we identify it, and why must it be chosen so precisely?

To understand what is happening here, it is helpful to separate the possible solution strings in Σ^n into two categories, which we will call solutions and nonsolutions. These sets can be described as follows:

$$A_0 = \{x \in \Sigma^n : f(x) = 0\}$$
$$A_1 = \{x \in \Sigma^n : f(x) = 1\}$$

where A_0 contains all the strings that evaluate to 0 when given as input to f, and A_1 has the strings we seek that evaluate to 1. In particular, we are interested in uniform superpositions over these two sets, defined as:

$$|A_0\rangle = \frac{1}{\sqrt{|A_0|}} \sum_{x \in A_0} |x\rangle$$
$$|A_1\rangle = \frac{1}{\sqrt{|A_1|}} \sum_{x \in A_1} |x\rangle$$

Importantly, we will assume from here on that A_0 and A_1 are both nonempty. However, we can still easily see what happens in the last two cases (all strings evaluate to 0, or all strings are solutions), as the former will return no solution, and in the latter case, any string is a solution. Since we are able to handle these special cases, we can safely continue our analysis under this assumption.

An key observation here is the fact that we can write the initialized uniform superposition as a linear combination of the states $|A_0\rangle$ and $|A_1\rangle$:

$$H^{\otimes n} |0^n\rangle = \sqrt{\frac{|A_0|}{N}} |A_0\rangle + \sqrt{\frac{|A_1|}{N}} |A_1\rangle$$

The key idea we utilize is this: **The state of the qubit register remains in the subspace spanned by $|A_0\rangle$ and $|A_1\rangle$ after every application of $\mathcal{G}$.** Thus, we would like to process the initialized state $H^{\otimes n} |0^n\rangle$ (with uniform amplitude across all N components) such that through successive iterations, we nudge the amplitude of the solution component $|A_1\rangle$ slightly higher, while lowering the amplitude of the nonsolutions $|A_0\rangle$. As we know, in a quantum state vector, the amplitude of any component squared represents the probability of seeing that component after measurement. Provided we find the suitable number of iterations t, we can push the amplitude of the state corresponding to the solutions arbitrarily high, and consequently see a high probability that this state is measured.

We can show that by applying $\mathcal{G}$ on the $|A_0\rangle$ and $|A_1\rangle$, we obtain the following linear combinations [6]:

Is the derivation of these expressions interesting content for an appendix, or do I just leave it?

$$\mathcal{G} |A_0\rangle = \frac{|A_0| - |A_1|}{N} |A_0\rangle + \frac{2\sqrt{|A_0| \cdot |A_1|}}{N} |A_1\rangle$$
$$\mathcal{G} |A_1\rangle = -\frac{2\sqrt{|A_0| \cdot |A_1|}}{N} |A_0\rangle + \frac{|A_0| - |A_1|}{N} |A_1\rangle$$

This clearly demonstrates that we are still in the span of $|A_0\rangle$ and $|A_1\rangle$ after an application of $\mathcal{G}$. Moreover, we can describe the action of $\mathcal{G}$ as a rotation matrix, and by once again following the example of Qiskit [6], we can also rewrite it as the square of a somewhat

simpler-looking matrix:

$$\begin{bmatrix} \frac{|A_0|-|A_1|}{N} & -\frac{2\sqrt{|A_0|\cdot|A_1|}}{N} \\ \frac{2\sqrt{|A_0|\cdot|A_1|}}{N} & \frac{|A_0|-|A_1|}{N} \end{bmatrix} = \begin{bmatrix} \sqrt{\frac{|A_0|}{N}} & -\sqrt{\frac{|A_1|}{N}} \\ \sqrt{\frac{|A_1|}{N}} & \sqrt{\frac{|A_0|}{N}} \end{bmatrix}^2$$

Now it is easier to see the similarity to the general form of a rotation matrix, with $\theta = \sin^{-1}\left(\sqrt{\frac{|A_1|}{N}}\right)$:

$$\begin{bmatrix} \sqrt{\frac{|A_0|}{N}} & -\sqrt{\frac{|A_1|}{N}} \\ \sqrt{\frac{|A_1|}{N}} & \sqrt{\frac{|A_0|}{N}} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Since squaring a rotation is the same thing as applying it twice, we return to the original rotation matrix for $\mathcal{G}$, and write it as:

$$\begin{bmatrix} \frac{|A_0|-|A_1|}{N} & -\frac{2\sqrt{|A_0|\cdot|A_1|}}{N} \\ \frac{2\sqrt{|A_0|\cdot|A_1|}}{N} & \frac{|A_0|-|A_1|}{N} \end{bmatrix} = \begin{bmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{bmatrix}$$

with θ defined as before. Thus, we have found a rotation matrix that describes the action of $\mathcal{G}$ on the subspace spanned by A_0 and A_1. This also means that the state of the qubit register after initialization can be described as:

$$H^{\otimes n}\,|0^n\rangle = \sqrt{\frac{|A_0|}{N}}\,|A_0\rangle + \sqrt{\frac{|A_1|}{N}}\,|A_1\rangle = \cos\theta\,|A_0\rangle + \sin\theta\,|A_1\rangle$$

So, each time we apply $\mathcal{G}$, the state of the register is rotated by 2θ, and we can find a closed-form expression for the state after t applications of $\mathcal{G}$. For brevity, we will henceforth refer to the initialized qubit register $H^{\otimes n}\,|0^n\rangle$ as $|u\rangle$.

$$\mathcal{G}^t\,|u\rangle = \cos((2t+1)\theta)\,|A_0\rangle + \sin((2t+1)\theta)\,|A_1\rangle$$

This expression makes sense intuitively: 2θ for each application of $\mathcal{G}$, and one additional θ to account for the state we started in.

Now recall that for any quantum state of the form $\alpha\,|A_0\rangle + \beta\,|A_1\rangle$, we measure a solution $x \in A_1$ with probability $|\beta|^2$. With the state in question $\mathcal{G}^t\,|u\rangle = \cos((2t+1)\theta)\,|A_0\rangle + \sin((2t+1)\theta)\,|A_1\rangle$, the probability of measuring a solution after t iterations is $\sin^2((2t+1)\theta)$. This is the probability we want to maximize in order to find a solution to the search problem with Grover's algorithm, and as such, we call $|A_1\rangle$ our *target state* [6].

Another consideration is that we would also like to minimize t, as this is the number of queries to f we have to make. Thus, to reach our target state with a probability close to 1 while minimizing t, we aim to find t such that $2(t+1)\theta \approx \frac{\pi}{2}$, since this is the smallest angle where the square of the sine function equals 1. In terms of t, we aim for $t \approx \frac{\pi}{4\theta} - \frac{1}{2}$. We cannot guarantee that this value of t is an integer, so we find the number of iterations

t by choosing $t = \lfloor \frac{\pi}{4\theta} \rfloor$. For the algorithm to work properly, it is also important that we do not iterate past the recommended iteration number, lest we avoid "overshooting" our solution by rotating past it. If we simply continue applying $\mathcal{G}$, we will in fact decrease the probability that the solution state is measured, and over time, we will oscillate between solutions and nonsolutions as we rotate the register state around the unit circle [6].

Up to this point, we have studied a general version of Grover's algorithm that can be applied to a number of search problems, ranging from those we know to have a single solution, a larger solution set of a fixed size, or even an unknown number of solutions. We have determined that an iteration number of $t = \lfloor \frac{\pi}{4\theta} \rfloor$ (and no more) will maximize the probability of finding one of these solutions, but what we must also consider is the fact that this way of setting iteration number t also depends on how many solutions there are, as θ depends on the size of the set A_1. Fortunately, since we are considering only the cryptographic application of using the algorithm to search for a single key from some symmetric cryptographic protocol, we can restrict ourselves to the cases where we know there is exactly one solution (or one cryptographic key).

Unique search, as this problem is formally known, is equivalent to the general unstructured search problem, with the additional promise that there is exactly one solution string x. Recall that in the general case, $\theta = \sin^{-1}\left(\sqrt{\frac{|A_1|}{N}}\right)$. Now that we have determined there is only a single solution, we know that $|A_1| = 1$. Thus, for large N, we see that $\theta = \sin^{-1}\left(\sqrt{\frac{1}{N}}\right) \approx \sqrt{\frac{1}{N}}$, since for smaller angles, the sine function starts to look more like the identity function [6]. If we use this approximation of θ in our previously determined expression $t = \lfloor \frac{\pi}{4\theta} \rfloor$, we get $t = \lfloor \frac{\pi}{4}\sqrt{N} \rfloor$. Since t represents queries to the function f, we see the quadratic improvement mentioned earlier: Rather than N queries, we are now on the order of the square root of that.

Practical Application and Limitations of Grover's Algorithm

At first glance, the improvements afforded by Grover's algorithm seem substantial. Though not a perfect algorithm in the sense that there is a probabilistic element and some chance of error[1], Grover's clearly has an advantage in runtime complexity over the classical strategy. Therefore, we identify that there is some trade-off between computational efficiency and certainty of outcome when choosing to use Grover's to tackle the unstructured search problem. In terms of correctness, it can be proven analytically that probability of success when using Grover's is always greater than or equal to $1 - \frac{1}{N}$ [6], meaning success is typically likely, particularly as N grows large. We are also free to run the algorithm in its entirety as many times as we choose, and if any one of them succeeds, it will identify a solution, with a high chance of doing so using fewer queries to f than its classical

[1] in contrast to the classical runtime we discussed, where the exhaustive search strategy leaves no chance of error whatsoever

counterpart. With the classical cost on the order of 2^{128} for AES-128, a naive calculation finds the new cost with Grover's square-root speedup to be around 2^{64}, perhaps with some small constant to account for overhead and setting up the quantum circuit. However, this estimation does not take into account a number of practical concerns and considerations.

First, Grover's algorithm, as described in Section 2.2.2, uses an oracle f, which in our case corresponds to the decryption algorithm in AES. A complicated scheme in its own right, AES must be implemented in the quantum hardware using computationally expensive gates and structures [4, 9], which we introduce and discuss in more detail in Section 2.4.1. This adds significant cost to the runtime of the algorithm [4], which the commonly cited Big O abstraction does not capture [10]. Exploring circuit designs of AES and their costs is outside of the scope of this project, but it is an active field of research [11, 12, 9] that will be crucial for full asymptotic analysis and practical implementation of Grover's algorithm applied to AES.

Secondly, as pointed out by Jaques, any realistic attack will require parallelization [4], and unfortunately, Grover's algorithm does not parallelize well. If we consider the full key space of N keys and partition it into P subsets, we can assign each of the P partitions to a machine responsible for a search space containing $\frac{N}{P}$ keys. With the square-root speedup over the classical method as discussed, we can now find the key in $\mathcal{O}(\sqrt{\frac{N}{P}})$ time. When comparing this to serial Grover's with a runtime of $\mathcal{O}(\sqrt{N})$, we find that parallelizing only reduces the runtime by a factor of $\sqrt{P}$, not P. Worse yet, the additional partitions add an overhead, with the total cost of operations going up to $P \cdot \mathcal{O}(\sqrt{\frac{N}{P}}) = \mathcal{O}(\sqrt{P \cdot N})$. Thus, in our case with AES-128, the bad parallelism of Grover means its cost is certainly higher than the naive estimation of 2^{64}.

The parallelization argument holds for any algorithm on the order of $\sqrt{N}$, and it turns out $\mathcal{O}(\sqrt{N})$ is the best we can do: For the general unstructured search problem, it has been proven that Grover's algorithm is *asymptotically optimal* [13, 14], meaning that as the search space grows, it is not possible to find an algorithm that performs better than on the order of $\sqrt{N}$ in the worst case. Despite its clear limitations, Grover's is thus still a contender to break the security of AES and similar symmetric encryption schemes by quantum computing, motivating the exploration we undertake in this project.

2.3 Quantum Error Correction

Justify why we are even discussing QEC.

Quantum info is inherently fragile. QEC combats this. Threshold theorems say that with a low enough ohysical error rate the logical error rate can be made arbitrarily small. Sounds great, but we incur an overhead. How badly does this scale? Measurement discretizes the noise. See image on phone.

2.3.1 Stable Quantum Memory

As we briefly touched upon, future quantum computers will certainly require some form of error correction, as quantum systems are prone to noise, causing logical errors during computation. Unlike in a classical computer where the only errors we consider are bit-flip errors (where 0 is interpreted as 1, or vice versa), quantum information can in principle be affected by a continuous space of errors. This turns out to be more manageable than it seems at first glance, however. To understand why, note that the set of Pauli matrices $\{\mathbb{I}_2, X, Y, Z\}$ forms a basis for $\mathbb{C}^{2\times 2}$ [15], meaning any error E on a single qubit can be decomposed linearly as:

$$E = \alpha\mathbb{I}_2 + \beta X + \gamma Z + \delta Y \quad \alpha, \beta, \gamma, \delta \in \mathbb{R}.$$

Since we have the relation $Y = iXZ$, we see that a Y-error can be expressed as a simultaneous X- and Z-error, which can be corrected with independent X and Z corrections. This discretization leaves only two types of errors we must correct that span all possible single-qubit errors (up to a global phase): X-errors, corresponding to bit-flips, and Z-errors, corresponding to phase-flips.

Having two types of fundamental errors to correct, compared to only one classically, in combination with quantum hardware being inherently more difficult to work with from an engineering perspective, has the unfortunate result of massively complicating the circuitry required for computation as soon as any practically viable error correction measures are introduced. The prevailing approach to managing this complexity is to encode logical information redundantly across many physical qubits, rather than attempting to perfect individual qubit quality to the point where errors become negligible, as the former is generally considered the easier path forward from an engineering standpoint [4]. This scheme, which can detect and correct errors without directly measurements, is what we call a *quantum error correcting code*. Exploring the challenges associated with encoding information in this way is the focal point of this project, which will be revisited in more detail in Section 2.4.1.

Toric Surface Code

The standard method of error correction that is used in all existing quantum computers built for fault-tolerance is the *surface code*. There is some inconsistency in the literature

In general, how do I say or cite statements about what the landscape in the field looks like right now?

about what constitutes a *toric code*, a *surface code*, and a *toric surface code*. We adhere to the nomenclature set by IBM [16], where toric codes exist in a grid across the surface of a torus with periodic boundary conditions, and surface codes and toric surface codes are used interchangeably to refer specifically to the portion of a torus grid laid flat with non-periodic boundary conditions.

Following the discovery of the toric code in 1997 [17], toric surface code has remained a prevalent error correcting architecture because it is based on simple principles and maintains geometric locality, meaning the only links required are between neighboring qubits, as opposed to also needing support for long-distance connections between qubits, which are more challenging from an engineering standpoint. In essence, surface code works by creating a grid of data qubits and measurement qubits interspersed and defining *stabilizers* that flag bit-flip (X) errors and phase-flip (Z) errors when they occur in specific locations. The error correcting capability lies in measuring the stabilizers across the grid repeatedly, decoding where errors have occured, and applying the appropriate correction based on the stabilizer measurement. The stabilizer formalism can in many ways be viewed as the quantum generalization of parity checks [18, 19], which are often used similarly in classical error correction to identify where and how to restore corrupted data.

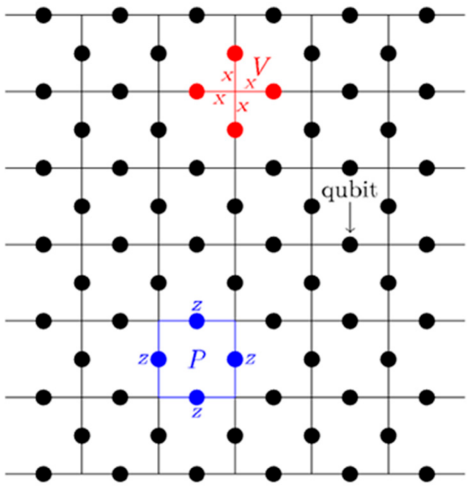


Figure 2.1: A surface code grid with physical data qubits located on the edges, demonstrating a plaquette (Z stabilizer) in blue and a vertex (X stabilizer) in red. Although only one of each is shown here, the entire grid is covered in these stabilizers in a repeating pattern.
Image source: Sayedsalehi et al. [20]
WILL BE REPLACED WITH MY OWN MORE EXPLANATORY DIAGRAM.

In Fig. 2.1, we show how taking four qubits surrounding a vertex or a square face in the grid (known as a *plaquette*) defines an X or Z stabilizer respectively. Check qubits for measurements in the stabilizers are a crucial element of quantum error correction specifically, as measuring the actual state of data qubits to identify data corruption would destroy their states, and thus the data itself.

I am aware "star" and "boundary" are not defined properly. I may slightly alter the flow here once I remake the graphic to rely on the visual more for the explanation.

The stabilizer formalism allows us to define algebraically the units described visually in Fig. 2.1 with two sets of commuting stabilizer generators. For each vertex v in the grid, we have an X stabilizer

$$S_v^X = \prod_{j \in \text{star}(v)} X_j,$$

which multiplies the Pauli X operator on the four edges touching v, or fewer at the grid boundaries. We similarly define a Z stabilizer for each plaquette p as

$$S_p^Z = \prod_{j \in \text{boundary}(p)} Z_j,$$

which multiplies the Pauli Z operator on the edges surrounding p. The groups formed by all S^X and S^Z respectively have elements that are all tensor products of Pauli operators and that commute. Together, each of these groups has a shared eigenspace constrained by

$$S_i \left|\psi\right\rangle = + \left|\psi\right\rangle \qquad \forall i,$$

where S_i represents a general stabilizer (either X or Z) applied to an appropriate vertex or plaquette, denoted by i. This eigenspace contains the legal code words, or in this case, the states corresponding to logical 0 and logical 1 (henceforth denoted by $\left|0\right\rangle_L$ and $\left|1\right\rangle_L$ respectively).

The Pauli operators satisfy the anticommutation relation $\sigma_j\sigma_k + \sigma_k\sigma_j = 2\delta_{jk}\mathbb{I}_2$, where σ_i are the Pauli operators according to the naming convention $\sigma_1 = X, \sigma_2 = Y, \sigma_3 = Z$, δ_{jk} is the Kronecker delta and $\mathbb{I}_2$ is the 2×2 identity matrix [21]. Thus, if an error operator E commutes with a stabilizer S_i, the eigenvalue of the surrounding qubits' state is unchanged:

$$S_i(E\left|\psi\right\rangle) = E(S_i\left|\psi\right\rangle) = +E\left|\psi\right\rangle.$$

However, if E and S_i anticommute, we will have $S_i(E\left|\psi\right\rangle) = -E(S_i\left|\psi\right\rangle)$ instead, which changes the eigenvalue of the local stabilizer state from $+1$ to -1.

At this point, we can do a measurement of the ancillary check qubits of the stabilizers to determine which ones violate our constraints and flag errors in their neighbors. Thinking back to the notion of parity checks, we consider the result of each stabilizer measurements a remapping of the parity check values 0 and 1 to $+1$ and -1 respectively. This is a very convenient mapping for us, because when we work with Z operators, ± 1 happen to be the eigenvalues of the eigenstates $\left|0\right\rangle$ and $\left|1\right\rangle$. Thus, ± 1 represent a way for us to access information about the state without measuring the state itself. In other words, if we measure the check qubits in the stabilizers and see $\left|0\right\rangle$, we know the data qubits are in the eigenstate $+1$, and we conclude no error has occurred among the neighboring

qubits of this stabilizer. However, if we see $|1\rangle$, we know the surrounding data qubits are in the eigenstate -1, and an error has occurred here. A measurement of an X-stabilizer is slightly more involved, but still unproblematic; the relation $HXH = Z$ allows us to simply apply H and do a measurement of Z-stabilizer instead.

Together, the eigenvalues s_i that we observe at each of the m stabilizer measurements form a string $(s_1, s_2, \ldots, s_{m-1}, s_m) \in \{\pm 1\}^m$ that we call the *syndrome* of the full state. It tells us about what errors are likely to have occurred and where, which we can use to determine what corrections we would like to apply to restore the logical state. It should be noted, however, that the syndrome cannot distinguish between no error and an even number of errors on the same stabilizer's neighbors, as both produce a $+1$ eigenvalue, leaving an undetectable error. This is a fundamental limitation of stabilizer codes generally, not specific to the surface code.

While conceptually simple, the toric surface code has a clear drawback in that it is very qubit-hungry, with estimates predicting a ratio around a 1000 to 1 of physical to logical qubits required for realistic error rates [4, 22]. This has sparked further study and innovation in order to uncover more favorable schemes in terms of resource efficiency [23].

Does this need any more sources when I'm trying to use it as a segue into BB codes?

Bivariate Bicycle Code: An Alternative to Toric Surface Code

The sentences in red imply we have already discussed Jaques' argument, which currently doesn't show up until way later in the text, but we need to go through the background to fully understand his argument. What is the best way to order this? One solution is just cutting them, thoughts?

The continued usage of the surface code at the current correction rates is one of the underlying assumptions in Jaques' predictions. Thus, it is interesting for the sake of our analysis to see how the prognosis for Grover's algorithm changes with a different choice of error correcting code. A few promising examples can be found in the *Low Density Parity Check* (LDPC) family of codes, of which the surface code is a special case. LDPC codes are stabilizer codes where each stabilizer acts on a fixed, small number of qubits, and each qubit participates in a fixed, small number of checks, both independent of the code size. For instance, in the surface code, each stabilizer acts on exactly four qubits (or fewer at boundaries) and each qubit participates in exactly two X-type and two Z-type checks. This property makes LDPC codes a strong contender for practical architectures, since stabilizer measurements remain local and manageable even as the code scales.

The *Bivariate Bicycle code*, also known as BB code, is one of these potential candidates to replace the surface code, also in the LDPC family [23]. Though similar to surface codes at a high level of abstraction, BB codes differ in that their stabilizers are not

geometrically local, and has therefore been called "surface code with extra long-distance checks" [24]. These checks allow for a number of advantages over the surface code, such as a higher threshold for error correction and lower qubit overhead [23].

We begin by defining the following shift matrices:

$$x = P_\ell \otimes \mathbb{I}_m, \quad y = \mathbb{I}_\ell \otimes P_m,$$

where $\mathbb{I}_m$ is the $m \times m$ identity matrix and P is a cyclic permutation matrix in the form $\sum_{i=1}^{\ell} |i\rangle \langle i+1|$, which maps $|i+1\rangle$ to $|i\rangle$ [24]. In addition, x and y satisfy $xy = yx$ and $x^\ell = y^m = \mathbb{I}_{\ell \times m}$. We use x and y to represent stepping through the grid in the x- and y-direction respectively.

Formally, the BB code is defined as a pair of $\ell m \times \ell m$ binary matrices $A(x, y), B(x, y)$ that are sums of our shift matrices x and y to integer powers. These matrix polynomials are then used to define parity check matrices H^X and H^Z:

$$H^X = [A|B], \quad H^Z = [B^\intercal|A^\intercal].$$

The dimension of H^X and H^Z is thus $\ell m \times 2\ell m$. For our purposes, $2\ell m = n$, the number of physical qubits, and $\ell m = \frac{n}{2}$, which is the number of vertex and plaquette stabilizers in the grid. We use these two matrices to index which qubits in the grid are to be acted on by stabilizers. Each row contains the qubit indices for a specific stabilizer, and each column is a binary indication of whether a specific qubit in the grid is part of that stabilizer. For instance, in BB code, each row will therefore have six 1s, as each stabilizer acts on six qubits, while the surface code in this notation would have four 1s per row (at most), because each stabilizer acts on only four qubits. The shifting mechanism defined by A and B is the mathematical way to describe the systematic construction of an evenly-spaced pattern repeating throughout the grid. With some appropriate indexing of the grid, we are able to specify the exact qubits to include in each stabilizer, uniquely defining a BB code.

Measuring a stabilizer corresponds to checking whether the associated parity condition set in the H^X and H^Z matrices is satisfied. Specifically, we require that

$$H^X (H^Z)^\intercal = 0.$$

This requirement ensures that the stabilizers commute, so that we can apply similar logic as with the standard surface code to identify the stabilizers for which this constraint does not hold, where the anticommutation relation will let us detect stabilizers in the -1 eigenstate. The syndrome that is produced informs the corrections we make as a result.

It is common to use the notation $[[n, k, d]]$ to specify the parameters of an error correcting code, where n is the number of physical qubits, k is the number of logical qubits we

encode, and d is the number of errors the code can correct. IBM, the developer of the BB code, has primarily considered a BB code it calls the *gross* code, after the term for a dozen dozen, known as a gross. Fittingly, its parameters are [[144, 12, 12]]: 144 physical qubits are required to encode 12 qubits, and the code can correct up to 12 errors without issue [23]. With the 1000 to 1 ratio needed for the toric surface code, the gross code (and BB codes in general) offer a clear improvement in this regard. As an additional plus, the connections between qubits in the BB code form a Tanner graph [25], which is a bipartite graph that can be separated into two disjoint sets. In this structure, nodes (qubits) within each set are only connected to members of the other, never to one another. This can be useful when physically constructing this architecture, since it allows the hardware to be split cleanly into two separate levels.

2.4 Fault-tolerant Quantum Computation

2.4.1 Universal Gate Sets

Assuming we have created some scheme for fault-tolerant storage of quantum data, we still face a number of significant challenges before we can perform any fault-tolerant computations. In particular, the *Eastin-Knill theorem* [26, 27] is commonly paraphrased as follows:

Do I include the formal statement anywhere here? Does it deserve its own appendix chapter? Or do I not include it at all?

The Eastin-Knill theorem (informal statement)

No quantum error correcting code can possess a universal set of transversal logical gates.

The full proof of this theorem, given by Eastin and Knill in their paper [26], relies on group theory and the algebraic properties of the error correcting codes themselves, and is far more rigorous and technical than we need for this project. For our purposes, it is enough to understand that the theorem places restrictions on the fault-tolerant quantum computation architecture. So far, we have not encountered the terms *transversal* and *universal*, but in this chapter, we will define them and argue why they are useful properties for a gate and gate set to have respectively, and thus why the Eastin-Knill theorem is such an important result in quantum computing.

Before we can fully understand what the theorem is saying, though, we need to delve further into how applying gates to quantum states forms a circuit. For this, we need the concept of a *unitary matrix*, or simply a unitary, which is a norm-preserving matrix $U \in \mathbb{C}^{N \times N}$, $N \in \mathbb{N}$, whose conjugate transpose $U^\dagger$ is also its inverse, meaning

$U^\dagger U = UU^\dagger = \mathbb{I}_N$. Mathematically, we can represent any quantum circuit on n qubits with a single unitary matrix.

$$U \in \mathbb{C}^{2^n \times 2^n}.$$

While this representation can certainly be useful, like in classical simulations with very few qubits, collapsing the circuit into one matrix in this way generally does not capture a lot of important behaviors and phenomena we are interested in when studying quantum error correction. In hardware, we never apply a single giant unitary gate; instead, as we have seen in the description of Grover's algorithm in Section 2.2.2, we decompose the computation into smaller, simpler gates. In doing so, we could in principle have our pick from a large selection of possible gates, but if for simplicity we restrict ourselves to implementing only a few different types, we would like the set of gates we choose to be *universal*. A universal gate set can approximate any such unitary U to an arbitrary degree of precision with a finite sequence of gates, an important property for our gate set to have, as without it, there will always be computations we cannot perform.

Furthermore, the Eastin-Knill theorem tells us that we cannot have a universal gate set in which every gate is *transversal*. To understand what this means, we need to introduce the concept of a *code block* (or *error-correcting patch*, interchangeably), which is what we collectively call the n physical qubits encoding a single logical qubit in a specific error-correcting code.

Transversality of a logical gate

Transversality is a property that a logical gate can have in a specific error-correcting code. A gate G_L acting on k logical qubits is transversal if it acts across the corresponding k code blocks by applying independent physical gates only between qubits at the same position i across blocks, with no coupling between positions $i \neq j$ within any single block:

$$G_L = \bigotimes_{i=1}^{n} G_i^{(1,2,\ldots,k)},$$

where $G_i^{(1,2,\ldots,k)}$ acts only on the i-th qubit in each of the k blocks.

The transversal property is illustrated for a single- and multi-qubit gate in Fig. 2.2. As the diagram shows, transversal gates ensure single-qubit errors do not propagate to multiple qubits within an error-correcting patch, making them inherently fault-tolerant: A single error occurring when applying the gate will produce at most one error for the error-correcting patch to correct. This is clearly a desirable property for gates in our chosen gate set to have, but the Eastin-Knill theorem describes a fundamental limitation,

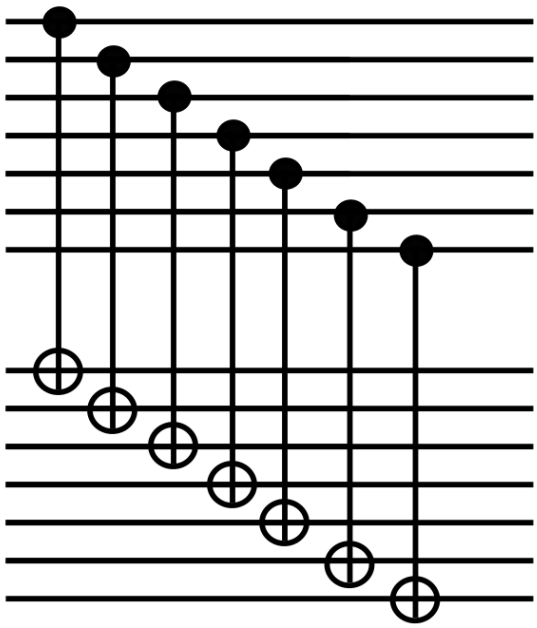


Figure 2.2: Illustration of transversality in a single- and multi-qubit gate. TO BE REPLACED WITH MY OWN DIAGRAM.

in that any gate set we choose capable of approximating arbitrary quantum circuits will contain at least one nontransversal gate. It is important to emphasize that transversality is not an inherent property of a gate, but rather a relationship between a gate and a particular error-correcting code. Thus, we would like to choose the error-correcting architecture and gate set in concert, such that as many commonly used gates as possible are transversal with respect to our chosen code.

Choosing a Gate Set: Clifford+T

Clifford+T is a common choice of universal gate set, fittingly comprised of *Clifford* and *T-gates*. Clifford gates are unitaries that map Pauli operators to Pauli operators under conjugation. Put simply, this trait unlocks some very desirable properties when performing computations in the quantum hardware (as well as when simulating classically, which will be explained in more detail in **??** in relation to our experiments). Recall from Section 2.3 that the stabilizers S_i are themselves tensor products of Pauli operators, and that the code space is defined by the eigenspace condition $S_i |\psi\rangle = +|\psi\rangle$. If we apply a Clifford gate C and ask whether the updated state $C|\psi\rangle$ still satisfies this condition, we find that the updated stabilizers $CS_iC^\dagger$ are again valid Pauli products simply by the definition of Clifford gates, meaning the code space is preserved. This is the algebraic reason Clifford gates are the natural choice for logical gates in stabilizer codes: They are compatible with the error-correcting architecture by construction.

In the codes we consider, we can verify gate by gate that the Clifford gates are transversal,

or that they decompose into independent single-qubit operations on physical qubits that implement the correct logical operation[2]. Together, the code space preservation and transversality that Clifford gates afford in the codes we use make them well-behaved: They stay in the same algebraic space; they have inherent fault-tolerance and a conceptually simple circuit design. The T-gates, however, given by:

I think we can verify transversality of gates in the surface code, but not necessarily in the BB code. Check this and write about it.

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix},$$

are a different story entirely. To understand why, note that conjugation by T does not preserve the Pauli group:

$$TXT^\dagger = \frac{1}{\sqrt{2}}(X + Y),$$

which is not a Pauli operator, but rather a linear combination of Paulis. This is the fundamental incompatibility: Since stabilizers are by definition Pauli products, any operation that preserves the stabilizer formalism must map Pauli products to Pauli products under conjugation. The T-gate violates this requirement, meaning it cannot be implemented by any manipulation of the stabilizer structure alone, and therefore cannot be transversal in the stabilizer codes we consider. This is not merely a consequence of Eastin-Knill, but the deeper algebraic reason behind it: The entire stabilizer framework only supports operations that are automorphisms of the Pauli group, and T is not one of them. However unwieldy T-gates may be, though, they are still a requirement for universality, and form a universal gate set along with the Clifford group. They necessitate complicated workarounds to be implemented fault-tolerantly, which we return to in **??**.

[2]Verifying the transversality of the Clifford gates in the codes we use is shown in Appendix B.

Chapter 3

Experiments and Analysis

3.1 General Limitations when Simulating Quantum Systems

In order to design and develop the technology necessary to begin implementing new error-correcting architectures, it is typically helpful to simulate how the components might work. In particular, the goal of this project is to simulate parts of the physical qubit setup needed for a working version of Grover's algorithm, a steppingstone on the way to fault-tolerant physical implementation. However, simulating the full fault-tolerant system with an error-correcting code of our choice turns out to be nearly impossible, even for the smallest nontrivial case of Grover's algorithm with only two input qubits.

To understand why this is so problematic, recall the expression for the unitary matrix defining an arbitrary quantum circuit with n input qubits:

$$U \in \mathbb{C}^{2^n \times 2^n}.$$

Given a gate set, we obtain U by *lifting* the individual gate matrices to the appropriate dimension $2^n \times 2^n$ and multiplying them in the right order. Specifically, in a quantum circuit, lifting a single-qubit gate $G \in \mathbb{C}^{2\times 2}$ acting on qubit k of an n-qubit system means constructing the corresponding $2^n \times 2^n$ unitary

$$\tilde{G} = \mathbb{I}_2 \otimes \cdots \otimes G \otimes \cdots \otimes \mathbb{I}_2,$$

where G appears in the k-th tensor factor. This approach preserves the mathematical structure of the full state at all times when applying a gate to only a few of the qubits, but it comes at a cost. The exponential scaling when n grows as we add qubits to the system means that we quickly face extremely large matrices when simulating anything beyond minimal examples or the logical abstractions of algorithms. This is bad news for us, as adding the error-correction architectures belonging to the families of primary interest that we wish to study will prevent us from being abel to simulate the circuit.

For instance, the smallest possible surface code is displayed in Fig. 3.1. It requires at least ten qubits, provided we reuse a single ancillary qubit for measurement rather than using eight individual ones. Each gate we would need in a simulation of Grover's with two logical qubits would be a $2^{20} \times 2^{20}$ matrix to account for the no fewer than 20 qubits in the system.

Add technical explanation/justification of memory requirements.

This also assumes none of the gates necessitate additional ancilla qubits to further drive up this cost, which, as we will see in Section 3.3 when we delve into the implementation of the T-gate, is not the case if we want a universal gate set.

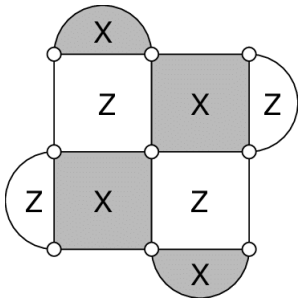


Figure 3.1: The smallest possible surface code consisting of nine data qubits. The indicated eight stabilizers could, in principle, be measured sequentially by a single additional qubit, or measured in parallel by an additional eight qubits. REMAKE MYSELF or ADD SOURCE TO BIBLIOGRAPHY.

3.1.1 Proof-of-concept Experiments with Smaller Codes

No published fault-tolerant physical implementation of Grover's algorithm exists at any scale, making it worthwhile to explore in detail how that might look. Aware of the limitations that come with simulating a large quantum system classically, two smaller error correcting codes, the *5-qubit code* and the *7-qubit Steane code*, were selected and implemented in code. These codes are defined and described in more detail in Appendix A.

Again, how do I deal with statements like this about the state of the field?

The 5-qubit code was a natural place to begin when searching for finding ways to simulate Grover's algorithm classically, since it is the smallest quantum error correcting code that can correct any type of error [15]. The code was implemented by explicitly constructing the encoding circuit and logical operators as $2^5 \times 2^5$ unitaries. While the quantum memory aspect worked as intended, it turns out that the 5-qubit code is not a good candidate for computation because in it, the Clifford gates are not transversal. This naturally led to considering the 7-qubit Steane code instead, another relatively small error correcting code where the Clifford gates do possess transversality, simplifying the implementation of certain logical operators. For this reason, the 7-qubit Steane code is a common choice in quantum error correction research settings today.

How much more detail/pseudocode/res do I include about this, if anything?

Simulating Toy Version of Grover's Algorithm

The circuit design for Grover's algorithm without any form for error correction is well-known, and is illustrated in Section 2.2.

How was it implemented? Add a diagram and maybe some pseudocode.

3.2 Utilizing Stabilizer Formalism as a Computational Tool

The exponential scaling of full unitary simulation, as described in Section 3.1, presents a fundamental barrier to classical simulation of large quantum circuits. In certain codes, however, we can at least partially circumvent the issue: For an n-qubit system, rather than storing and manipulating the full $2^n \times 2^n$ unitary, or even the state vector of 2^n complex amplitudes, a stabilizer state can be represented compactly by a set of n independent commuting Pauli operators $\{g_1, \ldots, g_n\}$ called *generators*. This uniquely specifies a state $|\psi\rangle$, which is the one joint $+1$ eigenstate of all g_i simultaneously. Thus, to recover the state vector from the generators, one simply asks which state satisfies $g_i |\psi\rangle = +|\psi\rangle$ for all i. To further illustrate how the stabilizer representation lets us recover the corresponding state vector uniquely, we will step through a small example.

Example: Recovering a state vector from the stabilizer representation

Consider the 3-qubit bit-flip code, with stabilizers $g_1 = ZZI$ and $g_2 = IZZ$, and logical operators $\bar{X} = XXX$, $\bar{Z} = ZZZ$. The generators alone define the codespace, spanned by $|\bar{0}\rangle = |000\rangle$ and $|\bar{1}\rangle = |111\rangle$, but do not yet specify which logical state we are in. To define a specific state, we augment the generator list with a logical operator. If we add $\bar{Z} = ZZZ$ as a third stabilizer, the unique joint $+1$ eigenstate of all three is $|000\rangle = |\bar{0}\rangle$, since we see that $ZZZ|111\rangle = -|111\rangle$, ruling out $|111\rangle = |\bar{1}\rangle$ as it is not a $+1$ eigenstate of the set. If we add $\bar{X} = XXX$ instead, the unique joint $+1$ eigenstate becomes $\frac{1}{\sqrt{2}}(|000\rangle + |111\rangle) = |\bar{+}\rangle$. In each case, the full state vector is recoverable from the generator list alone, with no ambiguity.

Have I previously made an argument for how many stabilizers are required for a given stabilizer code? If not I should do so or expand on this here.

The storage advantage is now clear: Each generator g_i is a tensor product of n single-qubit Paulis, requiring only $\mathcal{O}(n)$ bits to store, so the full generator list needs only $\mathcal{O}(n^2)$ bits, compared to $\mathcal{O}(2^n)$ complex numbers for the state vector.

We also have computational shortcuts when working with Clifford gates, which we recall from **??** are the gates that map Pauli operators to Pauli operators under conjugation. If we have a stabilizer state $|\psi\rangle$ with generators $\{g_1, \ldots, g_n\}$, we can apply a Clifford gate C to obtain a new state $C|\psi\rangle$. Rather than computing the full matrix product $C|\psi\rangle$ in a 2^n-dimensional space, we can take advantage of the stabilizer formalism and the fact that the generators uniquely determine the state to calculate the effect of C based on what happens to the generators. We want a new set of operators g_i' such that $g_i'(C|\psi\rangle) = C|\psi\rangle$. The following map achieves this:

$$g_i \mapsto C g_i C^\dagger.$$

We can verify that generators $g_i' = Cg_iC^\dagger$ defined in this way do indeed stabilize our updated state $C|\psi\rangle$, since it holds that:

$$g_i'C|\psi\rangle = Cg_iC^\dagger \cdot C|\psi\rangle = Cg_i|\psi\rangle = C|\psi\rangle .$$

This stabilizer update rule holds for any unitary matrix, but working with Clifford gates unlocks the advantage of ensuring $Cg_iC^\dagger$ is always another valid Pauli string, preserving the compact $\mathcal{O}(n)$-bit-per-generator storage throughout the entire computation. If conjugation by C could produce arbitrary matrices, we would be back to storing exponentially large objects and the efficiency gain would be lost.

To see the mechanics of the update rule, consider applying H to $|0\rangle$, stabilized by $g = Z$. We can simply update the stabilizer $HZH^\dagger = X$ and read off that the new state is stabilized by X, meaning it is the state $|+\rangle$. No matrix multiplication of a state vector is needed; the state update is captured in its entirety by relabeling one Pauli letter. In general, since a Clifford gate touches at most two qubits at a time, updating one generator costs $\mathcal{O}(n)$, relabeling at most two Pauli letters according to a fixed lookup table. Updating all n generators therefore costs $\mathcal{O}(n^2)$ in total, compared to the $\mathcal{O}(2^n)$ cost of full state vector simulation. This is the main result of the Gottesman-Knill theorem [28]: Any circuit composed entirely of Clifford gates can be simulated efficiently on a classical computer in the stabilizer representation.

Justify this.

3.2.1 Software Aids

The software package `PanQEC` [29] was used to visualize the geometry of several error correcting codes and to extract their logical operators. It stores stabilizer codes internally in *binary symplectic format*, in which each Pauli operator on n qubits is represented as a binary vector $v = (v_X \mid v_Z) \in \mathbb{F}_2^{2n}$. Here, the first n bits indicate which qubits carry an X factor and the last n bits indicate which carry a Z factor. A full stabilizer group with m generators is then stored as an $m \times 2n$ binary matrix, replacing what would otherwise require tracking m dense $2^n \times 2^n$ matrices. This yields a significant reduction in memory overhead and makes operations such as checking commutativity straightforward via the symplectic inner product.

Should I stylize PanQEC like this throughout or does it just look weird?

what is this

`PanQEC` also has functionality for determining logical operators of various types, which correct errors and perform small single-qubit computations.

I need to explore this more to fully understand how it works.

3.2.2 Simulating Stable Quantum Memory with PanQEC

A natural place to begin is with quantum memory. As a baseline and to verify the correctness of our `PanQEC` implementation, we do a quantum memory threshold simulation

where we compare the gross code to the surface code with identical logical qubit counts, in order to say something meaningful about the architectural trade-offs relevant to Jaques' assumptions. The error model applied is known as *depolarizing* error, and is the same across all experiments. In addition, all experiments use the BP-LSD decoder. We compare logical error rate as a function of physical error rate under Monte Carlo sampling.

more explanation?!!

more explanation needed?

Results

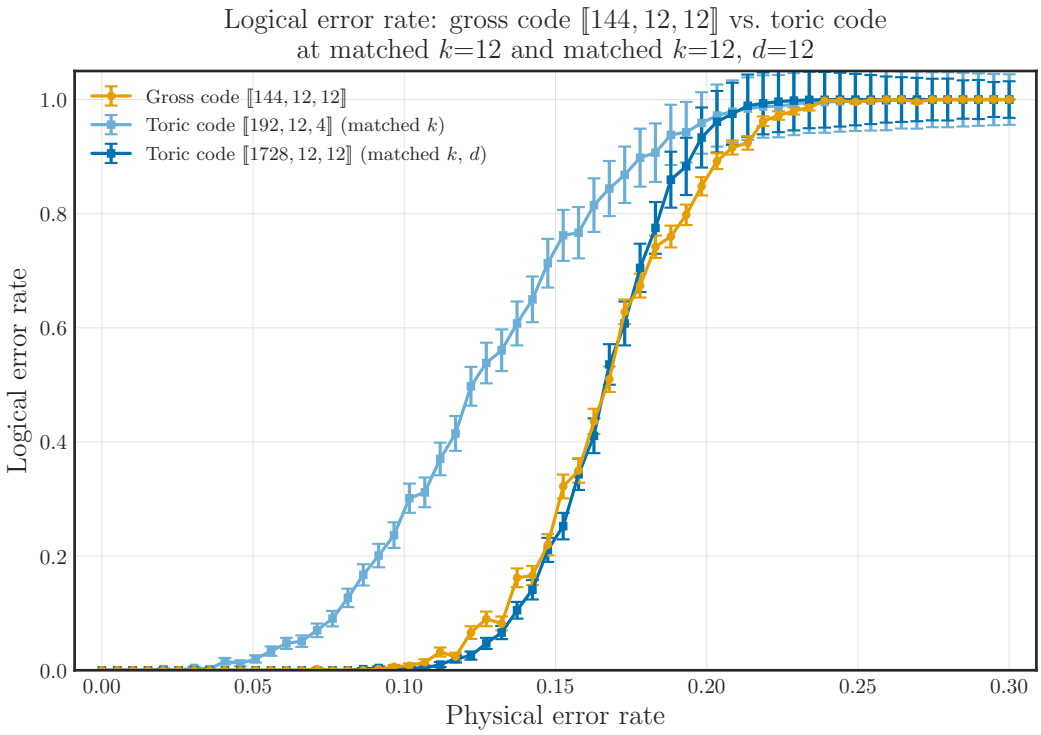


Figure 3.2: Quantum memory simulation results comparing the BB code (orange) against surface code implementations at identical logical qubit counts (light blue), as well as matching code distance (dark blue). Logical error rate is plotted as a function of physical error rate under Monte Carlo sampling.

At matched logical qubit count ($k = 12$), the gross code reaches a lower logical error rate at equivalent physical error rates, consistent with its more favorable qubit overhead ratio: The surface code requires $n = 192$ physical qubits to encode 12 logical qubits at distance $d = 4$, while the gross code achieves the same k at $d = 12$ using only $n = 144$. At matched k and d, we now see that the surface code is able to correct roughly the same number of errors as the gross code, which aligns with the theory given the equal code distance. However, there is now a huge disparity in the number of physical qubits required for the surface code to do this correction, as it requires $n = 1728$ physical qubits for equivalent error protection, a factor of twelve more than the gross code. This confirms the memory-level improvement that motivates our choice of BB code as an

alternative architecture, and identifies the physical error rate at which the gross code operates reliably as $p \approx 0.10$.

3.2.3 Simulating Transversal Logical Operators with PanQEC

Having established that the BB code offers an advantage over the surface code at the level of quantum memory, the natural next question is whether this advantage persists once we require the code to support logical computation. Jaques' argument, as discussed in **??**, identifies the cost of fault-tolerant gate implementation (specifically, the T-gate) as the dominant resource bottleneck, not memory overhead. The memory result therefore motivates but does not resolve the question of whether BB codes meaningfully improve the outlook for Grover's algorithm in practice. To begin probing this, we turn to the logical operators that `PanQEC` can simulate within the stabilizer formalism: the transversal Clifford gates.

Results

Do these experiments if possible, recreate summer student plots but with logical operators applied.

3.3 The Trouble with the T-gate

As established in **??**, the T-gate's incompatibility with the Pauli group under conjugation means it cannot be implemented by any manipulation of the stabilizer structure alone. This has a direct consequence for classical simulation: The moment a T-gate is introduced into a circuit, the stabilizer update rule $g_i \mapsto C g_i C^\dagger$ no longer produces a valid Pauli string, the compact $\mathcal{O}(n^2)$ stabilizer representation breaks down, and we must fall back to full state vector simulation at exponential cost. The efficient classical simulation guaranteed by the Gottesman-Knill theorem for Clifford circuits, discussed in Section 3.2, therefore does not extend to circuits containing T-gates.

As we have seen, applying the T-gate does not in any way reduce to stabilizer operations, it is not immediately obvious how to go from the PanQEC architecture to a working circuit.

What would we need to do to implement T-gates? (CCZ factory, magic state distillation)

3.3.1 Implemention

Show the minimal attempt (and explain where it breaks down if the experiment can't be successfully completed)

Add pseudocode for T-gate on individual error corrected qubits, not in a full circuit.

PSEUDOCODE GOES HERE!

3.4 Assessing the Quantum Threat

What does everything we've done so far tell us about Jaques' conclusion?

3.4.1 T-gate Cost as a Bottleneck

How T-gate count in the Grover circuit translates to physical resource requirements

Comparison across codes: surface code vs. BB code overhead per logical T-gate

Even with BB code's improved qubit ratio, does the T-gate problem go away?

3.4.2 Improvements with a Better Error Correcting Code

Revisiting Jaques' estimates under BB code assumptions

Where BB code helps (qubit overhead, threshold) and where it doesn't (T-gate implementation still hard)

Provided we need CCZ factories, we still need these in the BB codes. The main differences lie in other aspects. So it's good and bad news in a way, improving CCZ codes will create improvements everywhere, but it doesn't automatically make BB codes better.

Non-PanQEC diagrams of logical operator circuits across codes here???

3.4.3 Grover's Algorithm as a Practical Threat

What would need to change for Grover's to be viable?

Summarize the remaining gaps: T-gate implementation in BB code, circuit depth, parallelization?? i did already touch on parallelization but it is still relevant

Frame this constructively as directions for future work rather than just "it's still hard"

Chapter 4

Conclusion

BB codes reduce the overhead everywhere, but the fundamental bottleneck identified by Jaques remains, and closing it requires either better magic state distillation protocols or a gate set that avoids T-gates entirely in favor of something less costly. However, we know that it will be more costly than the Clifford gates in any case.

4.1 Future Work

Quantum storage:

Code distance scaling. How quickly does BB code improve when adding more qubits, compared to the surface code? This might already exist as a number.

Different noise models: Showing the comparison of BB code vs surface code for more complicated noise models.

Does the decoder we use for correcting errors matter at all and what is the efficiency/accuracy tradeoff? Is the BB code advantage robust under decoders?

4.2 Declaration of AI Usage

Rewrite this section into formal text.

I have used AI as an aid in coding and will probably continue to do so.
I use it to learn things sometimes and provide intuitive explanations where I otherwise just have equations to study.
I use it very minimally when writing, pretty much just when I can't remember a certain word or if I've used a word a lot and need a synonym (like a thesaurus).
I use it as a search engine when I have complicated search terms that Google can't find for me.
I ask it to format references from websites etc when there is not a downloadable option provided by the authors.

Appendices

Appendix A

Definitions of Smaller Error Correcting Codes Used

In addition to the aforementioned architectures that are viable options in the landscape of quantum technology today, this project also necessitated some smaller quantum codes to allow for easier simulation of circuitry.

A.1 The 5-Qubit Code

The 5-qubit code [15] is the smallest quantum error correcting code capable of correcting an arbitrary single-qubit error. Its parameters are $[[5, 1, 3]]$, meaning five physical qubits are used to encode a single logical qubit, with a code distance of three, allowing it to detect and correct any single-qubit error.

That five qubits is the minimum required for this purpose is not immediately obvious, but follows from a counting argument. Correcting an arbitrary single-qubit error on nn n qubits requires distinguishing between $3n + 1$ error syndromes (the three error types X, Y and Z on each of the n qubits, plus the no-error case). With $n - k$ stabilizer generators, the syndrome space has 2^{n-k} elements. For $k = 1$ logical qubit, we therefore require $2^{n-1} \geq 3n + 1$, which is first satisfied at $n = 5$.

source

source

The four stabilizer generators of the 5-qubit code are:

$$
\begin{aligned}
g_1 &= XZZX\mathbb{I} \\
g_2 &= \mathbb{I}XZZX \\
g_3 &= X\mathbb{I}XZZ \\
g_4 &= ZX\mathbb{I}XZ
\end{aligned}
$$

These generators, together with their products, define the codespace. The logical operators are $\bar{X} = XXXXX$ and $\bar{Z} = ZZZZZ$.

While the 5-qubit code is attractive for its minimality, it has a significant practical limitation: The Clifford gates are not transversal in this code. This means that implementing even computationally simple logical operations requires coupling physical qubits within the code block, removing the fault-tolerance guarantees that transversality would otherwise provide. For this reason, the 5-qubit code is well-suited as a proof-of-concept for quantum memory, but not as a platform for fault-tolerant computation, which motivated the move to the 7-qubit Steane code when performing experiments.

A.2 The 7-Qubit Steane Code

The 7-qubit Steane code [30, 31] is a $[[7, 1, 3]]$ code, encoding one logical qubit in seven physical qubits with code distance three, and is one of the most well-studied codes in quantum error correction. Like the surface code and the BB codes introduced in Section 2.3, it belongs to the *Calderbank-Shor-Steane* (CSS) family, meaning its stabilizer generators separate cleanly into an X-type set and a Z-type set.

source

The six stabilizer generators are:

$$
\begin{aligned}
g_1 &= \mathbb{I}\mathbb{I}\mathbb{I}XXXX \\
g_2 &= \mathbb{I}XX\mathbb{I}\mathbb{I}XX \\
g_3 &= X\mathbb{I}X\mathbb{I}X\mathbb{I}X \\
g_4 &= \mathbb{I}\mathbb{I}\mathbb{I}ZZZZ \\
g_5 &= \mathbb{I}ZZ\mathbb{I}\mathbb{I}ZZ \\
g_6 &= Z\mathbb{I}Z\mathbb{I}Z\mathbb{I}Z
\end{aligned}
$$

where g_1, g_2, g_3 are the X-type generators and g_4, g_5, g_6 are the Z-type generators. The logical operators are $\bar{X} = XXXXXXX$ and $\bar{Z} = ZZZZZZZ$.

The logical operators and encoding circuit for the Steane code were implemented explicitly in this project as $2^7 \times 2^7$ unitaries, making it the largest code for which full unitary simulation was tractable. The T-gate, as discussed in **??**, is not transversal in the Steane code, as is guaranteed by the Eastin-Knill theorem, and its implementation requires the magic state distillation techniques described there.

Appendix B

Verifying Transversality of Clifford Gates in Various Codes

B.1 Toric Surface Code

Do it.

B.2 BB Code

Check if this can be done easily, although I do think it is true.

Bibliography

[1] *Symmetric-key algorithms*. Wikipedia. URL: https://en.wikipedia.org/wiki/Symmetric-key_algorithm.

[2] National Institute of Standards et al. *Advanced Encryption Standard (AES)*. en. 2023-05-09 04:05:00 2023. DOI: https://doi.org/10.6028/NIST.FIPS.197-upd1. URL: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=936594.

[3] Ronald de Wolf. *Quantum Computing: Lecture Notes*. 2023. arXiv: 1907.09415 [quant-ph]. URL: https://arxiv.org/abs/1907.09415.

[4] Samuel Jaques. *Quantum Attacks on AES*. Presentation at the 26th International Conference on Cryptographic Hardware and Embedded Systems (CHES 2024). Available online: https://ches.iacr.org/2024/Jaques_CHES_2024.pdf. July 2024.

[5] Emma Storberg. *TEK5550 Mandatory Assignment 1*. Unpublished midterm exam. University of Oslo, 2025.

[6] Qiskit. *Grover's Algorithm | Understanding Quantum Information & Computation - Lesson 08*. Nov. 2023. URL: https://www.youtube.com/watch?v=hnpjC8WQVrQ.

[7] Wikipedia contributors. *Quantum logic gate — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Quantum_logic_gate&oldid=1278673600. [Online; accessed 4-March-2025]. 2025.

[8] Wikipedia contributors. *Quantum superposition — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Quantum_superposition&oldid=1275226563. [Online; accessed 4-March-2025]. 2025.

[9] Huinan Chen et al. *Quantum circuit for implementing AES S-box with low costs*. 2025. arXiv: 2503.06097 [quant-ph]. URL: https://arxiv.org/abs/2503.06097.

[10] Wikipedia contributors. *Big O notation — Wikipedia, The Free Encyclopedia*. [Online; accessed 27-January-2026]. 2026. URL: https://en.wikipedia.org/w/index.php?title=Big_O_notation&oldid=1333898274.

[11] Liao-Liang Jiang et al. "Constructing resource-efficient quantum circuits for AES." In: *Frontiers in Physics* Volume 13 - 2025 (2025). ISSN: 2296-424X. DOI: 10.3389/fphy.2025.1582819. URL: https://www.frontiersin.org/journals/physics/articles/10.3389/fphy.2025.1582819.

[12] Qun Liu et al. *Improved Quantum Circuits for AES: Reducing the Depth and the Number of Qubits*. Cryptology ePrint Archive, Paper 2023/1417. 2023. URL: https://eprint.iacr.org/2023/1417.

[13] Charles H. Bennett et al. "Strengths and Weaknesses of Quantum Computing." In: *SIAM Journal on Computing* 26.5 (Oct. 1997), pp. 1510–1523. ISSN: 1095-7111. DOI: 10.1137/s0097539796300933. URL: http://dx.doi.org/10.1137/S0097539796300933.

[14] Christof Zalka. "Grover's quantum searching algorithm is optimal." In: *Physical Review A* 60.4 (Oct. 1999), pp. 2746–2751. ISSN: 1094-1622. DOI: 10.1103/physreva.60.2746. URL: http://dx.doi.org/10.1103/PhysRevA.60.2746.

[15] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[16] IBM Quantum. *Other code families*. Accessed: 2025-01-28. IBM Quantum Learning. 2025. URL: https://quantum.cloud.ibm.com/learning/en/courses/foundations-of-quantum-error-correction/quantum-code-constructions/other-code-families.

[17] A.Yu. Kitaev. "Fault-tolerant quantum computation by anyons." In: *Annals of Physics* 303.1 (Jan. 2003), pp. 2–30. ISSN: 0003-4916. DOI: 10.1016/s0003-4916(02)00018-0. URL: http://dx.doi.org/10.1016/S0003-4916(02)00018-0.

[18] Wikipedia contributors. *Stabilizer code — Wikipedia, The Free Encyclopedia*. [Online; accessed 28-January-2026]. 2026. URL: https://en.wikipedia.org/w/index.php?title=Stabilizer_code&oldid=1332747113.

[19] Arthur Pesah. *The stabilizer trilogy I — Stabilizer codes*. Blog post. Jan. 31, 2023. URL: https://arthurpesah.me/blog/2023-01-31-stabilizer-formalism-1/ (visited on 01/28/2025).

[20] Samira Sayedsalehi et al. "Developing and Analyzing the Defect-Based Surface Codes Using Optimization Algorithms." In: *Quantum Reports* 7.2 (2025). ISSN: 2624-960X. DOI: 10.3390/quantum7020025. URL: https://www.mdpi.com/2624-960X/7/2/25.

[21] Wikipedia contributors. *Pauli matrices — Wikipedia, The Free Encyclopedia*. [Online; accessed 28-January-2026]. 2025. URL: https://en.wikipedia.org/w/index.php?title=Pauli_matrices&oldid=1327001669.

[22] Craig Gidney et al. "Yoked surface codes." In: *Nature Communications* 16.1 (May 2025). ISSN: 2041-1723. DOI: 10.1038/s41467-025-59714-1. URL: http://dx.doi.org/10.1038/s41467-025-59714-1.

[23] Sergey Bravyi et al. "High-threshold and low-overhead fault-tolerant quantum memory." In: *Nature* 627.8005 (Mar. 2024), pp. 778–782. ISSN: 1476-4687. DOI: 10.1038/s41586-024-07107-7. URL: http://dx.doi.org/10.1038/s41586-024-07107-7.

[24] Anton Brekke and Brage A. Trefjord. *Sommerprosjekt 2025*. Summer student project report on GitHub. [Online; accessed 28-Jan-2026]. 2025. URL: https://github.com/Bragit123/QEC/blob/main/rapport.pdf.

[25] Wikipedia contributors. *Tanner graph — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Tanner_graph&oldid=1296985959. [Online; accessed 28-January-2026]. 2025.

[26] Bryan Eastin and Emanuel Knill. "Restrictions on Transversal Encoded Quantum Gate Sets." In: *Physical Review Letters* 102.11 (Mar. 2009). ISSN: 1079-7114. DOI: 10.1103/physrevlett.102.110502. URL: http://dx.doi.org/10.1103/PhysRevLett.102.110502.

[27] Wikipedia contributors. *Eastin–Knill theorem — Wikipedia, The Free Encyclopedia*. [Online; accessed 26-January-2026]. 2025. URL: https://en.wikipedia.org/w/index.php?title=Eastin%E2%80%93Knill_theorem&oldid=1314440925.

[28] Daniel Gottesman. *The Heisenberg Representation of Quantum Computers*. 1998. arXiv: quant-ph/9807006 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9807006.

[29] panqec. *panqec: Simulation and visualization of quantum error correcting codes*. Version main. 2021. URL: https://github.com/panqec/panqec (visited on 02/18/2026).

[30] In: *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences* 452.1954 (Nov. 1996), pp. 2551–2577. ISSN: 1471-2946. DOI: 10.1098/rspa.1996.0136. URL: http://dx.doi.org/10.1098/rspa.1996.0136.

[31] "$[[7, 1, 3]]$ Steane code." In: *The Error Correction Zoo*. Ed. by Victor V. Albert and Philippe Faist. 2024. URL: https://errorcorrectionzoo.org/c/steane.